PALM-FL: Low-Payload Client-Summary Coordination and Resource-Aware Scheduling for Heterogeneous Mobile Federated Learning

Author Name(s) Omitted for Review

***Abstract*—Model heterogeneity is a practical obstacle to federated learning on mobile devices. In realistic deployments, clients differ in architecture, compute capability, memory budget, battery state, and link quality, making weight sharing through a single global model restrictive. This paper presents PALM-FL, a heterogeneous federated learning framework that keeps client architectures local and exchanges only compact clipped-and-noised client summaries. Each selected client uploads class-wise counts, latent means, and diagonal variances, and the server uses the resulting summary memory for budget-aware client selection rather than model aggregation. The zero-downlink stats-only path remains architecture-agnostic at the server, while a masked stats-transfer path uses the same compact memory for prototype-contrastive local regularization. The evaluation uses independent Dirichlet repartitions and trace-derived mobile profiles built from 49,760 public 4G, 5G, and WiFi bandwidth measurements. On MNIST, finite-noise random transfer reaches $56.11 \pm 3.99\%$ accuracy over three independent partitions, compared with $52.12 \pm 4.21\%$ for local-only and $51.59 \pm 4.74\%$ for stats-only. On a trace-calibrated CIFAR-10 seed-42 case study, random transfer reaches 29.47% accuracy, while mobile-aware transfer remains close to local/statistics baselines. Homogeneous FedAvg remains more accurate but requires a common architecture and full-model exchange. These results support PALM-FL as a low-payload, architecture-flexible coordination mechanism whose stats-only mode preserves local performance and whose transfer mode can help when participation coverage is broad enough.**

***Index Terms*—Federated learning, model heterogeneity, mobile computing, privacy accounting, communication efficiency, client scheduling.**

I. Introduction

FEDERATED learning allows multiple clients to train models collaboratively without centralizing raw data. Most federated learning methods, however, still assume that participating clients can train variants of a single shared model [3]–[5]. This assumption is increasingly difficult to justify in mobile systems. Smartphones, wearables, and embedded edge devices differ substantially in compute capability, memory budget, battery state, accelerator support, and link quality [7], [8]. In practice, many devices can host only lightweight client-specific architectures, while others depend on proprietary or hardware-tuned models that cannot be exposed or modified.

Model-heterogeneous federated learning has therefore become an important problem. Existing approaches address it in several ways. Supernet and submodel methods allow clients to train different-capacity models, but they still confine them to a common architecture family [9], [11]–[13]. Distillation-based methods permit broader heterogeneity, but often rely on proxy data, repeated logit exchange, or auxiliary public datasets [10], [14]–[17], [21]. Representation-sharing methods reduce the mismatch by introducing a common feature module, although they weaken full architectural autonomy [19], [20], [26], [27].

GEFL introduced generative transfer for heterogeneous federated learning, and GEFL-F reduced its cost by moving from image synthesis to feature synthesis through a shared extractor [1]. Even so, both approaches still require the exchange of global generative state. In mobile settings, that design raises two related questions. First, how should cross-client information be shared when communication, latency, energy, and privacy are all constrained? Second, how much of the server state must be coupled to client architectures?

This paper addresses these questions through PALM-FL. The central idea is to move the main server-side coordination path away from generator broadcast and toward clipped-and-noised client-summary release. Each client keeps its encoder, adapter, and prediction head local. Instead of uploading parameters or downloading synthetic features, the client computes compact class-wise counts and latent summaries, clips and perturbs them, and sends only the sanitized summaries to the server. The server maintains a global summary memory and uses it to guide budget-aware client selection under explicit uplink, downlink, energy, and round-time constraints. The default operating mode therefore requires no model download and no architecture alignment across clients.

The contributions of this work are as follows.

1) We propose PALM-FL, a heterogeneous mobile federated learning framework in which the server coordinates training from compact clipped client summaries rather than architecture-specific parameter alignment.
2) We develop a clipped-and-noised summary-release mechanism for class-wise counts, latent means, and diagonal variances that supports zero-downlink operation in the main training path.
3) We introduce a resource-aware client scheduler that balances data utility, class coverage, staleness, battery state, communication, and time under explicit mobile budgets.
4) We evaluate the method with independent Dirichlet repartitions on MNIST and trace-derived mobile profiles built from public 4G, 5G, and WiFi bandwidth measurements.
5) We evaluate heterogeneous MNIST and CIFAR-10 clients and show that stats-only closely tracks, rather than substantially improves over, local-only while maintaining zero downlink and small uplink.
6) We include a masked prototype-contrastive stats-transfer variant and show that it improves MNIST accuracy under

random participation, while mobile-aware CIFAR-10 transfer remains a low-payload but scheduler-sensitive operating point.

7) We include homogeneous FedAvg, FedProto-style, and FedMD-style references that expose the trade-off among accuracy, communication, architecture autonomy, proxy data, and prototype exchange.
8) We report per-architecture fairness summaries and show that scheduler choice defines a real trade-off: random participation improves utility, while mobile-aware selection controls operating cost.

II. Related Work

A. Homogeneous federated learning and mobile FL systems

FedAvg remains the standard baseline for homogeneous federated learning, while FedProx, SCAFFOLD, and adaptive local aggregation variants improve robustness under statistical and systems heterogeneity [3]–[6]. Surveys on constrained and IoT federated learning emphasize that realistic edge deployments must manage client-side limitations in computation, memory, energy, and network bandwidth rather than optimize only for accuracy [7], [8]. PALM-FL follows that systems view directly.

B. Model-heterogeneous federated learning

The heterogeneous-FL literature can be grouped into several families. A first family scales a shared supernet or global architecture into client-specific submodels, as in FjORD, HeteroFL, DepthFL, and NeFL [9], [11]–[13]. These methods are effective when clients remain within the same architecture family, but they do not provide full architectural autonomy. A second line of work addresses heterogeneity through distillation, ensembling, or proxy-data transfer, including AvgKD, FedDF, FCCL, FedMD, FedGH, and pFedHR [10], [14]–[17], [21]. Prototype-sharing methods such as FedProto exchange class prototypes instead of gradients or complete weights [18]. These methods permit broader heterogeneity, but often depend on proxy data, repeated logit exchange, auxiliary public datasets, or unmasked class prototypes. A third family shares a representation module or auxiliary model, as in LG-FedAvg, federated mutual learning, FedClassAvg, and continual horizontal FL for heterogeneous data [19], [20], [26], [27]. These approaches reduce part of the mismatch, but they still impose a partially shared representation.

C. Generator-centered transfer and privacy

Generator-centered FL has been explored through federated GANs, conditional feature generators, and data-free distillation [22]–[25]. GEFL is the closest precursor to this paper. It uses a federated generator to transfer global knowledge across clients with different models, and GEFL-F reduces the cost of that idea by shifting from image generation to feature generation through a shared extractor [1]. The base paper also highlights the remaining issues: privacy leakage through memorization, performance degradation as client count grows, and the resource overhead of the shared generative state [1]. Representation release can also be vulnerable to inversion-style attacks [33]–[35]. PALM-FL addresses the same problem family by using compact latent moments rather than generator broadcast as the main exchange primitive.

Table I summarizes representative methods discussed in the base paper and related literature. PALM-FL is the only row that combines client-specific heterogeneous architectures, no shared extractor, zero-downlink operation in the main path, and explicit budget-aware scheduling.

III. System Model and Objective

We consider a cross-device federated learning system with client set $\mathcal{K} = \{1, \ldots, K_{\text{cl}}\}$. Client k stores a private dataset

$$\mathcal{D}_k = \{(x_i, y_i)\}_{i=1}^{N_k}, \qquad y_i \in \{1, \ldots, C\}, \tag{1}$$

and uses a private model

$$f_k(x; \theta_k) = H_k(A_k(E_k(x))), \tag{2}$$

where E_k is a private encoder, A_k is a private latent adapter, and H_k is a private prediction head. The latent representation is

$$z_i^{(k)} = A_k(E_k(x_i)) \in \mathbb{R}^d, \tag{3}$$

where the latent dimension d is shared, but the client backbones and heads may differ completely.

Each client is associated with a device profile

$$\psi_k = (b_k, c_k, e_k, a_k), \tag{4}$$

where b_k is available bandwidth, c_k is normalized compute throughput, e_k is remaining battery energy, and $a_k \in [0, 1]$ is an availability probability. In round t, the server selects a subset $\mathcal{S}_t \subseteq \mathcal{K}$ subject to a maximum participation size M and the system budgets

$$\sum_{k \in \mathcal{S}_t} U_k^t \le B_{\text{up}}, \tag{5}$$

$$\sum_{k \in \mathcal{S}_t} D_k^t \le B_{\text{down}}, \tag{6}$$

$$\sum_{k \in \mathcal{S}_t} E_k^t \le B_{\text{eng}}, \tag{7}$$

$$\max_{k \in \mathcal{S}_t} T_k^t \le B_{\text{time}}, \tag{8}$$

where U_k^t, D_k^t, E_k^t, and T_k^t denote the predicted uplink, downlink, energy, and round-time cost for client k in round t.

The overall objective is to coordinate local training and scheduling under these constraints while limiting information exposure from the released summaries:

$$\begin{aligned} \min_{\{\theta_k\}, \pi} \quad & \sum_{t=1}^{T} \sum_{k \in \mathcal{S}_t} \mathcal{L}_k(\theta_k; \mathcal{D}_k) \\ \text{s.t.} \quad & \mathcal{S}_t = \pi(\mathcal{H}_t, \mathcal{M}_t), \quad |\mathcal{S}_t| \le M, \\ & \text{the mobile budgets hold in every round.} \end{aligned} \tag{9}$$

Here $\mathcal{H}_t$ is the server-side state and $\mathcal{M}_t$ is the collection of clipped-and-noised client summaries available at round t.

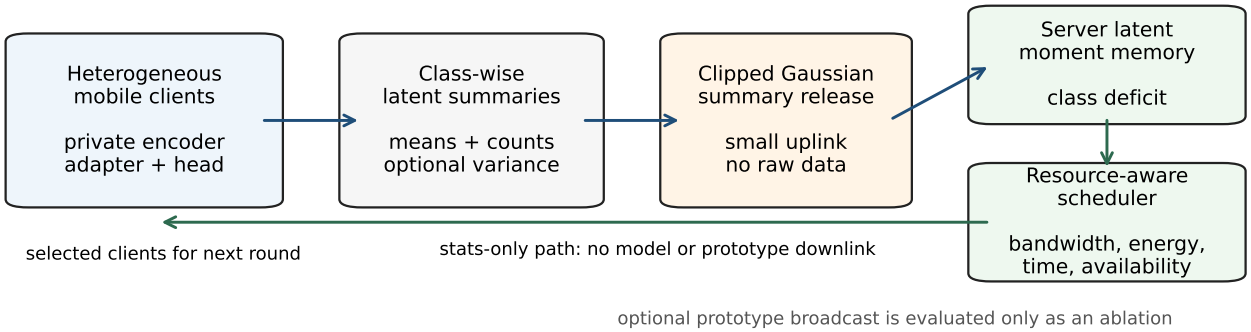

Fig. 1. Overview of PALM-FL. Each client keeps its encoder, adapter, and prediction head private, extracts compact class-wise latent moments, clips and perturbs them before upload, and is selected by a resource-aware scheduler subject to communication, energy, and round-time budgets. In the default stats-only mode, the server uses the latent memory only for scheduling and does not broadcast models or prototypes back to the clients.

TABLE I
DESIGN-SPACE COMPARISON WITH REPRESENTATIVE HETEROGENEOUS FEDERATED LEARNING METHODS. "PARTIAL" INDICATES THAT CLIENTS CAN DIFFER IN CAPACITY OR HEAD STRUCTURE BUT STILL REMAIN WITHIN A SHARED SUPERNET, EXTRACTOR, OR REPRESENTATION FAMILY. PALM-FL STILL ASSUMES A SHARED LATENT DIMENSION THROUGH PRIVATE ADAPTERS IN THE SUMMARY-RELEASE PATH.

Method family	Client-specific arch.	Proxy data	Shared module	Default payload	Zero-downlink	Mobile-aware
Homogeneous FL [3], [4], [6]	No	No	Yes	weights	No	No
Submodel / supernet FL [9], [11]–[13]	Partial	No	Yes	submodel	No	No
Proxy-data distillation FL [10], [14]–[17], [21]	Yes	Often yes	No	logits / models	No	No
Representation-sharing FL [19], [20], [26], [27]	Partial	No	Yes	representation weights	No	No
GEFL [1]	Yes	No	No	generator	No	No
GEFL-F [1]	Partial	No	Yes	feature generator	No	No
PALM-FL	**Yes**	**No**	**No**	**summaries**	**Yes**	**Yes**

IV. PALM-FL METHOD

A. Client-side summary extraction

Each selected client computes class-wise counts and latent moments from its local minibatches. For class c, define

$$n_{k,c} = \sum_{(x,y)\in\mathcal{D}_k} \mathbb{I}[y=c], \tag{10}$$

$$\mu_{k,c} = \frac{1}{\max(n_{k,c},1)} \sum_{(x,y)\in\mathcal{D}_k} z(x)\,\mathbb{I}[y=c], \tag{11}$$

$$v_{k,c} = \frac{1}{\max(n_{k,c},1)} \sum_{(x,y)\in\mathcal{D}_k} z(x)\odot z(x)\,\mathbb{I}[y=c] - \mu_{k,c}\odot\mu_{k,c}, \tag{12}$$

where $z(x) = A_k(E_k(x))$ and $\odot$ denotes elementwise multiplication. The implementation releases the class mean, diagonal variance, noisy count, and a count-derived availability mask. The stats-only mode uses these summaries only on the server side; it does not broadcast prototypes or models to clients. In this default path, the class-deficit term is count-driven, so the current results should not be interpreted as isolating the marginal value of latent means over clipped-and-noised class-count balancing.

This choice is practical for mobile devices. The uploaded state scales with the latent dimension d and the class count C, and is independent of the parameter count of the client model. The server therefore needs no architecture-specific merger, no backbone alignment, and no global decoder.

B. Clipped-and-noised moment release

Before transmission, PALM-FL clips the client summaries and perturbs them with Gaussian noise. Let $M_k = [\mu_{k,1};\ldots;\mu_{k,C}]$ and $V_k = [v_{k,1};\ldots;v_{k,C}]$ stack the class-wise mean and diagonal-variance vectors, and let $\mathbf{n}_k = (n_{k,1},\ldots,n_{k,C})$. The privacy unit in the current implementation is the released client-summary object, not an individual raw training example: two summary inputs are treated as adjacent when their clipped released statistic differs by at most the stated ℓ_2 bound. With clipping thresholds S_μ, S_v, and S_n, the released summaries are

$$\widehat{M}_k = \mathrm{clip}_2(M_k, S_\mu) + \Xi_k, \tag{13}$$

$$\widehat{V}_k = \mathrm{clip}_2(V_k, S_v) + \Psi_k, \tag{14}$$

$$\widehat{\mathbf{n}}_k = \mathrm{clip}_2(\max(\mathbf{n}_k, 0), S_n) + \boldsymbol{\zeta}_k, \tag{15}$$

where

$$\mathrm{vec}(\Xi_k) \sim \mathcal{N}(0, \sigma^2 S_\mu^2 I), \tag{16}$$

$$\mathrm{vec}(\Psi_k) \sim \mathcal{N}(0, \sigma^2 S_v^2 I), \tag{17}$$

$$\boldsymbol{\zeta}_k \sim \mathcal{N}(0, \sigma^2 S_n^2 I). \tag{18}$$

The rows of $\widehat{M}_k$ and $\widehat{V}_k$ define $\widehat{\mu}_{k,c}$ and $\widehat{v}_{k,c}$, and the entries of $\widehat{\mathbf{n}}_k$ define $\widehat{n}_{k,c}$. Released counts are truncated to be nonnegative, and the mask is computed by thresholding the noisy count. In our implementation, privacy is accumulated over participating rounds with a participation-counted zCDP-style accountant [30], [31]. The reported privacy values should be interpreted as accounting values for clipped client-summary release; the learned latent summaries are not accompanied by a sample-level sensitivity proof, and stronger sample-level certification would require per-example clipping and accounting.

C. Server-side latent memory and deficit estimation

Let $\beta \in [0,1)$ denote the server exponential moving average factor. For each class c, the server ignores client payloads whose count-derived mask is false and then forms the round-level sufficient statistics

$$\widetilde{N}_c^{(t)} = \sum_{k\in\mathcal{S}_t} \widehat{n}_{k,c}, \tag{19}$$

$$\widetilde{\mu}_c^{(t)} = \frac{\sum_{k\in\mathcal{S}_t} \widehat{n}_{k,c}\widehat{\mu}_{k,c}}{\max\left(\widetilde{N}_c^{(t)}, \epsilon\right)}, \tag{20}$$

$$\widetilde{q}_c^{(t)} = \frac{\sum_{k\in\mathcal{S}_t} \widehat{n}_{k,c}\left(\widehat{v}_{k,c} + \widehat{\mu}_{k,c}\odot\widehat{\mu}_{k,c}\right)}{\max\left(\widetilde{N}_c^{(t)}, \epsilon\right)}, \tag{21}$$

$$\widetilde{v}_c^{(t)} = \max\left(\widetilde{q}_c^{(t)} - \widetilde{\mu}_c^{(t)}\odot\widetilde{\mu}_c^{(t)}, v_{\min}\right). \tag{22}$$

It then updates the class count, prototype mean, and diagonal variance with exponential memory, matching the implemented aggregator:

$$N_c^{(t)} = \beta N_c^{(t-1)} + (1-\beta)\widetilde{N}_c^{(t)}, \tag{23}$$

$$\bar{\mu}_c^{(t)} = \beta\bar{\mu}_c^{(t-1)} + (1-\beta)\widetilde{\mu}_c^{(t)}, \tag{24}$$

$$\bar{v}_c^{(t)} = \beta\bar{v}_c^{(t-1)} + (1-\beta)\widetilde{v}_c^{(t)}. \tag{25}$$

Previously unseen classes are initialized from the first observed $\widetilde{\mu}_c^{(t)}$ rather than from the zero vector. The global class deficit vector is

$$\boldsymbol{\delta}_c^{(t)} = \frac{\max_j N_j^{(t)} - N_c^{(t)}}{\sum_{m=1}^{C}\left(\max_j N_j^{(t)} - N_m^{(t)}\right) + \epsilon}, \tag{26}$$

which assigns larger weight to underrepresented classes. In stats-only, the server does not broadcast $\bar{\mu}_c^{(t)}$ to clients. Instead, the deficit vector $\boldsymbol{\delta}^{(t)}$ is consumed only by the scheduler, which avoids additional downlink and keeps the main learning path architecture-agnostic. This also scopes the role of the latent means: in stats-only, class counts drive the deficit term, while the latent means and variances maintain the server memory used by the optional transfer path.

D. Resource-aware scheduling

For each candidate client, the server estimates communication and compute cost from device metadata. The predicted time and energy are modeled as

$$T_k^t = \frac{8U_k^t}{b_k} + \frac{8D_k^t}{b_k} + \alpha_T\frac{L_kP_k}{c_k}, \tag{27}$$

$$E_k^t = \alpha_U(U_k^t + D_k^t) + \alpha_E L_k P_k, \tag{28}$$

where P_k is the model size in MB-equivalent parameters, L_k is the estimated number of local steps, and α_T, α_U, and α_E map bandwidth and compute to time and energy.

The scheduler assigns utility to client k in round t using data availability, estimated label diversity, staleness, battery state, recent loss, and deficit coverage:

$$\begin{aligned} u_k^t = {} & w_d s_k + w_\ell \ell_k + w_r r_k + w_b q_k + w_v \nu_k \\ & + w_c\left\langle \mathbf{1}[\widehat{h}_k > 0], \boldsymbol{\delta}^{(t)}\right\rangle - \lambda_u\frac{U_k^t}{B_{\text{up}}} - \lambda_d\frac{D_k^t}{B_{\text{down}}} \\ & - \lambda_e\frac{E_k^t}{B_{\text{eng}}} - \lambda_t\frac{T_k^t}{B_{\text{time}}}, \end{aligned} \tag{29}$$

where s_k is a normalized sample-count score, ℓ_k is a normalized recent-loss score, r_k is a staleness score, q_k is remaining battery fraction, ν_k is an estimated label-diversity score, and $\widehat{h}_k \in \mathbb{R}^C$ is the most recent clipped-and-noised client count vector available to the server. The scheduler does not require raw label histograms before selection; clients with no prior accepted summary use a zero class-coverage vector until they participate. The selected client set is obtained from

$$\begin{aligned} \max_{\mathcal{S}\subseteq\mathcal{K}} \quad & \sum_{k\in\mathcal{S}} u_k^t \\ \text{s.t.} \quad & \sum_{k\in\mathcal{S}} U_k^t \le B_{\text{up}}, \quad \sum_{k\in\mathcal{S}} D_k^t \le B_{\text{down}}, \\ & \sum_{k\in\mathcal{S}} E_k^t \le B_{\text{eng}}, \quad \max_{k\in\mathcal{S}} T_k^t \le B_{\text{time}}, \quad |\mathcal{S}| \le M. \end{aligned} \tag{30}$$

In our implementation, this problem is solved with a ratio-greedy selection rule that orders clients by utility normalized by budget consumption.

E. Local optimization

The default local objective combines classification loss with a covariance regularizer on the latent representation:

$$\mathcal{L}_k = \mathbb{E}_{(x,y)\sim\mathcal{D}_k}\left[\mathrm{CE}(f_k(x), y)\right] + \lambda_{\text{cov}}\Omega(z), \tag{31}$$

where $\Omega(z)$ penalizes deviations of the latent covariance from the identity matrix. If $Z \in \mathbb{R}^{B\times d}$ is the minibatch latent matrix, then

$$\Omega(z) = \frac{1}{d}\sum_{j=1}^{d}(\Sigma_{jj} - 1)^2 + \frac{1}{d(d-1)}\sum_{i\neq j}\Sigma_{ij}^2, \tag{32}$$

with $\Sigma = \frac{1}{B}(\widetilde{Z}^\top\widetilde{Z})$ and $\widetilde{Z}$ the standardized latent matrix. This regularizer stabilizes the latent space while remaining fully local.

F. Masked prototype-contrastive transfer

PALM-FL also supports a stats-transfer variant in which the server broadcasts a compact masked prototype package derived from the latent memory. The package contains available class prototypes, diagonal variances, class counts, and class priors. A class is marked available only when its sanitized count exceeds a configurable threshold, which prevents absent or noise-dominated classes from polluting the prototype bank.

Algorithm 1 PALM-FL round with optional masked stats-transfer

Require: client set $\mathcal{K}$, latent memory $(\bar{\mu}^{(t-1)}, N^{(t-1)})$, budgets $(B_{\text{up}}, B_{\text{down}}, B_{\text{eng}}, B_{\text{time}})$

compute deficit vector from (26)
estimate $(U_k^t, D_k^t, E_k^t, T_k^t)$ for all eligible clients
select $\mathcal{S}_t$ with the budget-aware scheduler
for all $k \in \mathcal{S}_t$ in parallel **do**
  train the private model with loss (31)
  compute $(n_{k,c}, \mu_{k,c}, v_{k,c})$ for all classes c
  privatize $(n_{k,c}, \mu_{k,c}, v_{k,c})$ with clipped Gaussian release
  upload $(\widehat{n}_{k,c}, \widehat{\mu}_{k,c}, \widehat{v}_{k,c}, m_{k,c})$
aggregate uploaded moments into $(\bar{\mu}^{(t)}, \bar{v}^{(t)}, N^{(t)})$ and class mask $m^{(t)}$
if stats-transfer is enabled and warm-up has ended **then**
  export masked prototype package to selected clients in the next round
update the scheduler state with observed loss, participation, and device cost

When a prototype package is available after warm-up, selected clients add two lightweight local transfer terms. The first aligns real-batch latents with their own available class prototype:

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{(x,y)} \left[\mathbf{1}[m_y = 1] \left\| \frac{z(x)}{\|z(x)\|_2} - \frac{\bar{\mu}_y}{\|\bar{\mu}_y\|_2} \right\|_2^2 \right], \quad (33)$$

where m_y is the server mask for class y. The second treats the available prototypes as class anchors. Define

$$s_c(x) = \frac{\langle \text{norm}(z(x)), \text{norm}(\bar{\mu}_c) \rangle}{\tau}. \quad (34)$$

The prototype-contrastive cross-entropy is

$$\mathcal{L}_{\text{pCE}} = \mathbb{E}_{(x,y):m_y=1} \left[-\log \frac{\exp(s_y(x))}{\sum_{c:m_c=1} \exp(s_c(x))} \right]. \quad (35)$$

The resulting local objective is

$$\mathcal{L}_k^{\text{transfer}} = \mathcal{L}_k + \lambda_a \mathcal{L}_{\text{align}} + \lambda_p \mathcal{L}_{\text{pCE}}. \quad (36)$$

This differs from unmasked prototype-broadcast ablations: unavailable classes are explicitly masked and the contrastive term uses other available global prototypes as negatives. It adds a small downlink proportional to Cd, but still avoids model, gradient, generator, or decoder broadcast.

G. Analysis

Proposition 1 (Communication complexity)**.** *If class-wise means, diagonal variances, counts, and masks are uploaded, then the per-round uplink of client k in stats-only is $\mathcal{O}(2Cd+C)$ scalars and does not depend on the parameter count of f_k.*

Proof: The uploaded state contains one d-dimensional mean vector, one d-dimensional diagonal variance vector, one scalar count, and one mask bit per class, which requires $2Cd + \mathcal{O}(C)$ scalars. This is $\mathcal{O}(Cd)$ for fixed moment order and class count. ■

Algorithm 2 Budget-aware client selection

Require: candidate clients $\mathcal{A}_t$, utility scores u_k^t, per-client costs $(U_k^t, D_k^t, E_k^t, T_k^t)$, maximum size M

sort candidates by utility normalized by budget consumption
initialize $\mathcal{S}_t \leftarrow \emptyset$
for all k in sorted order **do**
  if $|\mathcal{S}_t| = M$ **then**
    break
  if adding client k violates an uplink, downlink, energy, or time budget **then**
    continue
  add k to $\mathcal{S}_t$
if $\mathcal{S}_t = \emptyset$ **then**
  return the best single feasible client
return $\mathcal{S}_t$

Proposition 2 (Architecture-agnostic server state)**.** *The server-side state of PALM-FL depends only on latent dimension d, class count C, clipped count summaries, and scalar resource metadata. It does not require layer-wise parameter aggregation, weight matching, or architecture-specific adapters.*

Proof: The server receives only sanitized moments and device metadata. The latent-memory update and the scheduler operate on these quantities only; no step depends on the internal parameterization of E_k, A_k, or H_k. ■

Proposition 3 (Client-summary Gaussian accounting)**.** *Suppose each released client-summary statistic is clipped to its stated summary-level sensitivity and perturbed with Gaussian noise calibrated to that sensitivity. Then each statistic satisfies the standard Gaussian-mechanism guarantee for the clipped summary under the summary-level adjacency relation described above. For a client that participates in R_k rounds with r Gaussian releases per round, the accountant used here tracks*

$$\rho_k = \frac{R_k r}{2\sigma^2}, \qquad \varepsilon_k(\delta) = \rho_k + 2\sqrt{\rho_k \log(1/\delta)}. \quad (37)$$

Remark 1. *The experiments reported here use a participation-counted zCDP-style accountant. This is sufficient for comparing operating modes under the same release mechanism, but it is not a complete sample-level privacy certificate. The reported ε values are also large, so the mechanism should be interpreted as clipped-and-noised summary release with accounting rather than strong end-to-end differential privacy.*

V. Experimental Setup

A. Datasets, clients, and models

We evaluate PALM-FL on MNIST and CIFAR-10 [28], [29] under heterogeneous cross-device partitioning. Both datasets are divided across ten clients using a Dirichlet partition with $\alpha = 0.3$, which creates strong label skew while preserving client diversity. Each client uses one of four lightweight architectures: `small_cnn`, `wide_cnn`, `tiny_resnet`, or `tiny_mobilenet`. All models on a dataset share only the adapter output dimension: $d = 48$ for MNIST and $d = 64$ for CIFAR-10.

TABLE II
PRIVACY-ACCOUNTING PARAMETERS USED IN THE FINITE-NOISE SUMMARY-RELEASE ROWS. "N/A" MEANS THE METHOD IS NOT ACCOUNTED UNDER PALM-FL'S CLIPPED SUMMARY-RELEASE MECHANISM.

Mode	Released objects	Accounting unit	Interpretation
local-only	none	N/A	no summary-release accountant
stats-only	counts, means, variances; $S_\mu = S_v = 4$, $S_n = 200$, $\sigma = 0.5$, $\delta = 10^{-5}$	client summary	high ε; not sample-level DP
stats-transfer	same upload plus masked prototype package downlink	client summary	high ε; not sample-level DP
FedAvg	model weights	N/A	not private by this accountant
FedMD-style	proxy logits	N/A	not private by this accountant

B. Trace-derived mobile profiles

To move beyond a purely synthetic mobile profile, we derive the bandwidth component of the scheduler profiles from the public Mobile Access Bandwidth in Practice release [2]. The release contains anonymized representative samples from 49,760 Android bandwidth tests across 4G, 5G, WiFi 4, WiFi 5, and WiFi 6. We construct a deterministic ten-client profile file by selecting technology-stratified bandwidth quantiles from the released CSV files. The resulting trace profiles span 1.2–681.2 Mbps in this run and are used by PALM-FL, FedAvg, FedProto-style, and FedMD-style experiments through the same scheduler interface. Compute throughput, battery capacity, recharge, and per-step energy remain model parameters because the bandwidth trace does not include direct model-training energy measurements. Thus, the reported time and energy values are trace-calibrated operating estimates, not phone power-meter measurements.

C. Training, baselines, and metrics

MNIST uses 15 rounds with 3 warm-up rounds and a 0.4 participation rate. The MNIST rows report mean and sample standard deviation over seeds 42, 43, and 44, with the client partition seed and architecture seed both varied with the training seed. CIFAR-10 uses 20 rounds, 4 warm-up rounds, evaluation every five rounds, and a full final evaluation. Because full trace-calibrated CIFAR-10 independent reruns are substantially more expensive in this implementation, the CIFAR-10 rows in Table III are reported as a completed seed-42 trace case study; fixed-split three-seed CIFAR artifacts are retained in the reproducibility bundle but are not used for the independent-split claim.

We compare local-only, stats-only, mobile-scheduled stats-transfer, and random-scheduled stats-transfer. We also include three references. Homogeneous FedAvg uses a shared `small_cnn`; FedProto-style keeps heterogeneous clients but disables PALM stat-head and prototype-contrastive terms; FedMD-style uses a public proxy split and exchanges averaged proxy logits. We report accuracy, macro-F1, uplink, downlink, trace-calibrated predicted round time, predicted energy, and privacy loss from the participation-counted zCDP-style accountant where that accountant is part of the method. Rows without this mechanism are marked as not DP-accounted rather than assigned an ε value. Finite-noise summary-release rows use $S_\mu = S_v = 4.0$, $S_n = 200$, $\delta = 10^{-5}$, and $\sigma = 0.5$. Accuracy is averaged over client-local models under a low-round, low-participation, non-IID protocol, so the MNIST values are not intended to match fully centralized or long-horizon MNIST training.

D. Implementation and reproducibility

All experiments are executed from the same implementation tree and stored as completed run artifacts. The package includes the trace-profile builder, independent-split experiment driver, FedMD-style baseline, aggregation scripts, curated CSV files, regenerated figures, and per-client final metrics for fairness analysis. The public artifact repository is available at https://github.com/Devchandrasen/PALM-FL-TMC-Artifact.

VI. RESULTS

A. Operating-mode comparison

Table III presents the operating-mode comparison. On MNIST, random-scheduled stats-transfer reaches $56.11 \pm 3.99\%$ accuracy over three independent partitions, compared with $52.12 \pm 4.21\%$ for local-only and $51.59 \pm 4.74\%$ for stats-only. Mobile-scheduled transfer remains close to stats-only, which means the mobile policy is better interpreted as a cost-control policy than as an accuracy maximizer. On the CIFAR-10 seed-42 trace case study, random transfer reaches 29.47% accuracy, while mobile transfer remains near the local/statistics baselines. This confirms a consistent scheduler-sensitive pattern without overstating mobile transfer as a universal accuracy improvement.

B. Reference baselines

Table IV reports the baseline references. FedAvg remains the high-accuracy reference because it assumes a single shared architecture and full-model exchange. On MNIST it reaches $95.98 \pm 1.35\%$ accuracy but uploads and downloads 1.5996 MB per selected-client set. Its lower predicted time and energy in this table should not be read as a like-for-like systems win over PALM-FL, because FedAvg uses a homogeneous `small_cnn` workload rather than the heterogeneous client models used by PALM-FL. FedProto-style has the same compact payload scale as PALM transfer but lacks the stat-head and contrastive terms. FedMD-style provides a true heterogeneous distillation reference with a public proxy split and 0.0781 MB proxy-logit upload and downlink on MNIST. It underperforms PALM random transfer in this setting, indicating that proxy-logit exchange alone is not a sufficient replacement for the latent-memory design. CIFAR-10 FedProto-style and FedMD-style trace-derived reruns are not yet complete, so they are omitted rather than mixed with older fixed-split artifacts.

C. Communication, privacy, and fairness

Figure 3 separates the communication and privacy dimensions for the heterogeneous PALM operating modes. The stats-only mode keeps downlink at zero, while transfer adds

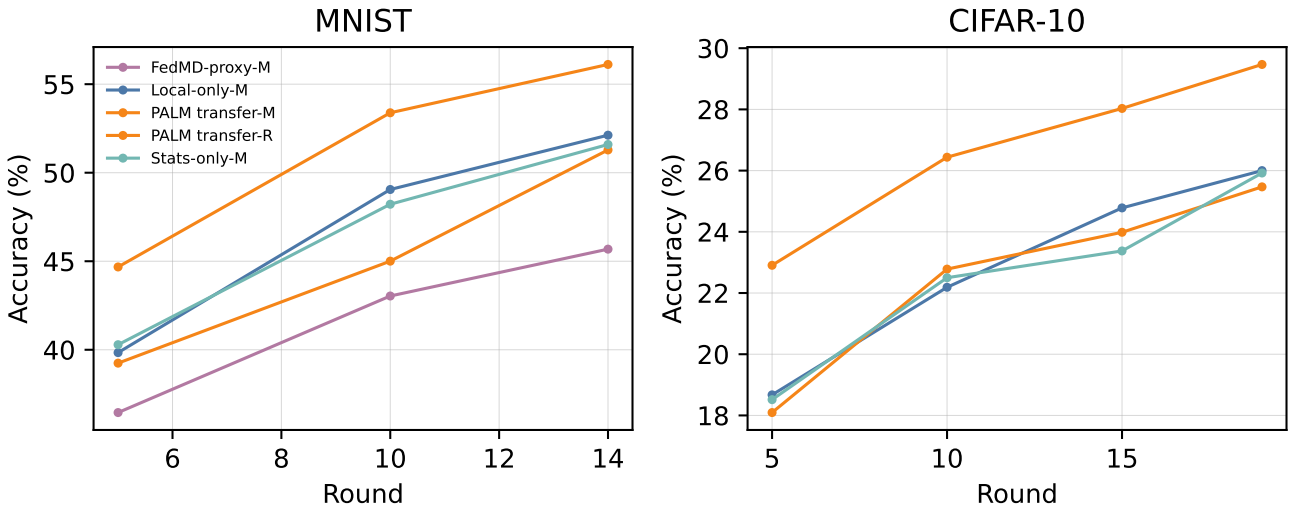

Fig. 2. Learning curves from the curated trace-derived independent-split runs. MNIST curves are averaged over seeds 42, 43, and 44; CIFAR-10 curves use the completed trace-calibrated seed-42 case study.

TABLE III
TRACE-DERIVED PALM-FL OPERATING COMPARISON. ACCURACY IS REPORTED IN %. MNIST ROWS REPORT MEAN ± SAMPLE STANDARD DEVIATION OVER THREE INDEPENDENT REPARTITIONS. CIFAR-10 ROWS REPORT THE COMPLETED TRACE-CALIBRATED SEED-42 CASE STUDY.

Dataset	Mode	Acc. (%)	F1	Up. (MB)	Down. (MB)	Time (s)	Energy (J)	Summary $\varepsilon_{\max}$
MNIST	local-only	52.12±4.21	0.4159±0.0485	0.0000	0.0000	16.24	**5.92**	–
MNIST	stats-only	51.59±4.74	0.4104±0.0526	0.0148	0.0000	16.25	5.98	140.69
MNIST	stats-transfer-M	51.29±3.85	0.4056±0.0422	0.0148	0.0150	16.59	6.88	140.69
MNIST	stats-transfer-R	56.11±3.99	0.4459±0.0461	0.0148	0.0150	23.27	6.87	100.92
CIFAR-10	local-only	26.00	0.1589	0.0000	0.0000	58.98	10.36	–
CIFAR-10	stats-only	25.92	0.1563	0.0197	0.0000	58.99	10.36	194.34
CIFAR-10	stats-transfer-M	25.47	0.1487	0.0197	0.0199	59.51	10.45	194.34
CIFAR-10	stats-transfer-R	**29.47**	**0.1787**	0.0197	0.0199	59.51	**9.82**	129.58

"M" and "R" denote mobile-aware and random schedulers. Privacy values are client-summary accounting values, not sample-level end-to-end DP certificates. "–" means the summary-release accountant is not applicable to that row.

TABLE IV
TRACE-DERIVED BASELINE REFERENCES. MNIST ROWS REPORT MEAN ± STANDARD DEVIATION OVER THREE INDEPENDENT REPARTITIONS. THE CIFAR-10 FEDAVG ROW REPORTS THE COMPLETED SEED-42 TRACE CASE STUDY.

Dataset	Baseline	Acc. (%)	F1	Up. (MB)	Down. (MB)	Time (s)	Energy (J)	Summary $\varepsilon_{\max}$
MNIST	FedAvg	95.98±1.35	0.9594±0.0137	1.5996	1.5996	8.30	3.46	–
MNIST	FedProto-style	50.23±3.97	0.3959±0.0436	0.0148	0.0150	16.25	6.89	140.69
MNIST	FedMD-style	45.68±4.18	0.3436±0.0524	0.0781	0.0781	11.96	3.55	–
CIFAR-10	FedAvg	59.39	0.5464	1.7292	1.7292	13.58	9.27	–

"–" means the method is not accounted under PALM-FL's clipped summary-release mechanism; it should not be read as a privacy guarantee.

a compact prototype package. FedAvg is omitted from this plot because its full-model payload is roughly two orders of magnitude larger and is reported separately in Table IV. The privacy panel also clarifies that the current ε values are high: PALM should be read as a clipped-and-noised summary-release mechanism, not as a strong sample-level DP system.

Figure 5 summarizes per-architecture final accuracy for PALM random transfer. The architecture-level spread is substantial, especially on CIFAR-10, which shows why heterogeneous-FL papers should report client-group behavior rather than only averaged endpoint accuracy. In the MNIST three-seed run, `tiny_resnet` clients benefit most from the random-transfer operating point, while `wide_cnn` clients remain weaker under the non-IID splits used here.

D. Discussion

Four points follow directly from the evidence. First, upload-only stats-only remains the safest low-communication operating mode because it tracks local-only while keeping downlink at zero; it does not demonstrate a strong federated learning gain. Second, transfer helps most when the scheduler provides

Fig. 3. Reported round-level communication and client-summary privacy values from the curated trace-derived runs. stats-only has zero downlink; stats-transfer adds a compact masked prototype package.

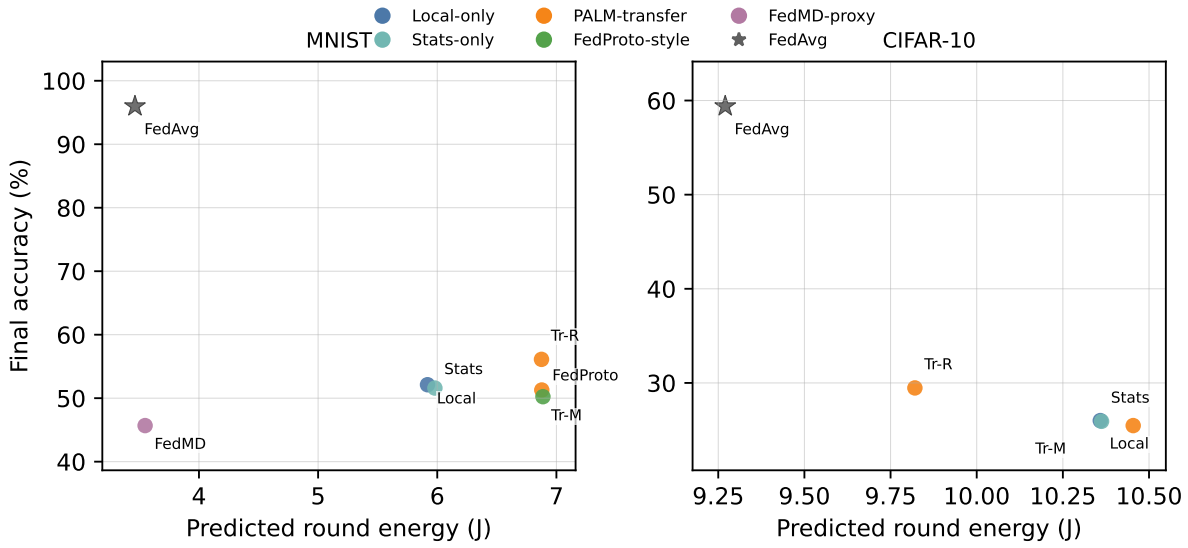

Fig. 4. Accuracy–energy operating frontier from the curated trace-derived runs. Each dataset is plotted separately to distinguish heterogeneous PALM-FL operating modes from the homogeneous FedAvg reference.

broader client and class coverage; random participation improves MNIST and CIFAR-10 seed-42 utility, whereas mobile-aware transfer stays near the lower-cost local/statistics operating point. Third, FedAvg confirms the accuracy available when the problem is changed to homogeneous full-model sharing, but it is not a like-for-like heterogeneous systems comparison. Fourth, the trace-derived profile changes the cost scale relative to a synthetic profile, so claims based only on uniformly sampled bandwidth would be misleading.

Table V reports completed MNIST and CIFAR-10 ablations over seeds 42, 43, and 44. The histogram-only rows are important because they show that clipped-and-noised class counts alone can produce competitive utility under random participation. The mean-only rows are also diagnostic, but they use noisy availability rather than the full count-deficit signal. The no-noise rows are explicitly non-private diagnostics, and the count-only transfer row removes prototype vectors from the downlink package. These rows improve claim isolation, but they do not prove that latent moments are the dominant source of utility: on CIFAR-10, random histogram-only and random

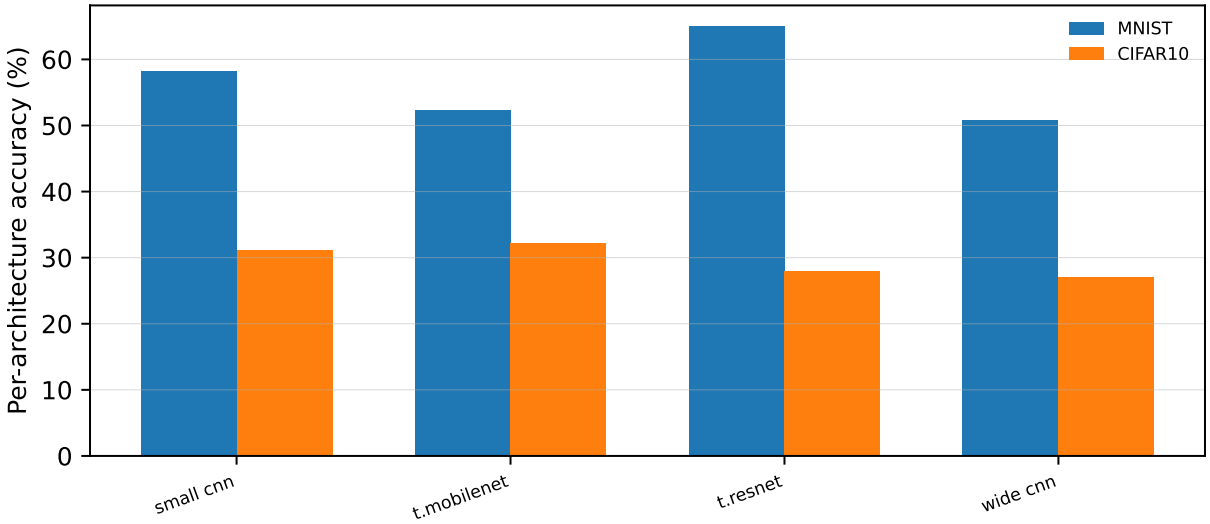

Fig. 5. Per-architecture final accuracy for PALM random transfer. MNIST bars aggregate three independent repartitions; CIFAR-10 bars use the completed seed-42 trace case study.

mean-only are close, and no-deficit random transfer remains close to the original random-transfer case-study endpoint.

VII. Limitations and Future Work

The mobile validation is trace-derived for bandwidth only; real-device training latency and battery power measurements remain necessary for deployment claims. The time and energy values should therefore be interpreted as operating estimates from a cost model, not as measured phone energy. The MNIST main table uses independent repartitions, but the primary CIFAR-10 operating-mode and baseline comparison still has only a completed seed-42 trace case study, so a full three- or five-seed CIFAR-10 trace rerun is needed for stable main-result claims. The present study also uses ten clients and two image datasets; larger cross-device populations and additional mobile workloads are needed. The privacy accountant is participation-counted zCDP over clipped client summaries and does not certify end-to-end sample-level DP. The ε values are high, and full-model/proxy-logit baselines are not DP-accounted, so the claim should remain limited to privacy-scoped summary release. The new ablations in Table V are three-seed diagnostics on both datasets, but they show that counts, participation coverage, and scheduler choice explain much of the observed behavior; therefore, the value of latent moments beyond clipped-and-noised class-count balancing remains only partially isolated. Finally, FedAvg, FedProto-style, and FedMD-style references improve baseline coverage, but reproducing stronger submodel and distillation baselines such as HeteroFL/FjORD, NeFL, GEFL/GEFL-F, and FedDF/FedGH would further strengthen the empirical comparison.

VIII. Conclusion

PALM-FL addresses heterogeneous mobile federated learning through clipped-and-noised client-summary release and resource-aware client selection. The framework avoids architecture-specific parameter aggregation, sanitizes compact class-wise summaries before upload, and makes participation decisions under explicit communication, time, and energy budgets. The experiments use independent MNIST repartitions and public trace-derived mobile bandwidth profiles. The results show that stats-only preserves the low-payload local-training operating point, while masked stats-transfer can improve utility when scheduler coverage is broad enough. PALM-FL is therefore best positioned as an architecture-flexible, low-payload, privacy-scoped coordination mechanism that complements, rather than replaces, high-accuracy homogeneous full-model FL.

Appendix A
Prototype-Transfer Variants

For completeness, the prototype-transfer family augments (31) with prototype alignment and prototype cross-entropy terms:

$$\mathcal{L}_k^{\text{proto}} = \mathcal{L}_k + \lambda_a \mathcal{L}_{\text{align}} + \lambda_p \mathcal{L}_{\text{pCE}}. \tag{38}$$

The unmasked prototype-broadcast ablation was useful for controlled comparison but underperformed in the retained experiments. The stats-transfer path used in this paper differs by masking unavailable classes and using the prototype bank as a contrastive anchor set.

References

[1] H. Kang, S. Cha, and J. Kang, "GeFL: Model-agnostic federated learning with generative models," *IEEE Trans. Mobile Comput.*, vol. 25, no. 4, pp. 4493–4505, Apr. 2026, doi: 10.1109/TMC.2025.3592483.

[2] X. Yang *et al.*, "Mobile access bandwidth in practice: Measurement, analysis, and implications," in *Proc. ACM SIGCOMM*, 2022, pp. 114–128, doi: 10.1145/3544216.3544237.

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.

[4] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, 2020, pp. 429–450.

TABLE V
ABLATION STUDY ISOLATING COUNT, LATENT, NOISE, TRANSFER, AND SCHEDULER EFFECTS. COMPLETED MNIST AND CIFAR-10 ABLATION ROWS USE SEEDS 42, 43, AND 44.

Dataset	Mode	Scheduler	Accuracy	Macro-F1	Upload MB	Download MB	Predicted time s	Predicted energy J	Epsilon/accounting	Interpretation
MNIST	histogram-only	mobile	48.96±5.09	0.3858±0.0485	0.0002	0.0000	11.55	3.34	61.70	counts-only scheduler control; three MNIST seeds
MNIST	histogram-only	random	56.51±3.46	0.4527±0.0391	0.0002	0.0000	22.77	6.73	45.58	counts-only random-participation control; three MNIST seeds
MNIST	mean-only	mobile	50.80±2.76	0.4064±0.0315	0.0075	0.0000	16.25	5.92	102.88	latent means with noisy availability; three MNIST seeds
MNIST	mean-only	random	55.70±4.24	0.4449±0.0536	0.0075	0.0000	22.77	6.73	74.61	latent means under random participation; three MNIST seeds
MNIST	stats-only no-noise	mobile	51.32±5.10	0.4060±0.0552	0.0148	0.0000	16.25	5.98	non-private diagnostic	non-private summary diagnostic; three MNIST seeds
MNIST	transfer no-noise	mobile	52.17±2.89	0.4163±0.0332	0.0148	0.0150	17.96	4.48	non-private diagnostic	non-private transfer diagnostic; three MNIST seeds
MNIST	count-only transfer	mobile	52.09±2.84	0.4129±0.0421	0.0002	0.0003	20.88	4.87	61.70	transfer package without prototypes; three MNIST seeds
MNIST	Stats-only	mobile/no_deficit	51.02±4.09	0.4056±0.0466	0.0148	0.0000	16.25	5.92	140.69	full-summary scheduler without released-count deficit; three MNIST seeds
MNIST	stats-transfer-M	mobile/no_deficit	50.55±3.47	0.4023±0.0403	0.0148	0.0150	16.59	6.01	140.69	mobile transfer without released-count deficit; three MNIST seeds
MNIST	stats-transfer-R	random/no_deficit	55.49±3.93	0.4426±0.0450	0.0148	0.0150	23.27	6.87	100.92	random transfer without released-count deficit; three MNIST seeds
CIFAR-10	histogram-only	mobile	26.96±2.52	0.1738±0.0281	0.0002	0.0000	38.50	12.63	81.89	counts-only scheduler control; three CIFAR-10 seeds
CIFAR-10	histogram-only	random	32.28±2.59	0.2132±0.0270	0.0002	0.0000	79.12	19.10	56.07	counts-only random-participation control; three CIFAR-10 seeds
CIFAR-10	mean-only	mobile	28.56±3.19	0.1861±0.0323	0.0100	0.0000	41.09	16.29	138.85	latent means with noisy availability; three CIFAR-10 seeds
CIFAR-10	mean-only	random	32.73±2.25	0.2163±0.0211	0.0100	0.0000	79.13	19.11	92.96	latent means under random participation; three CIFAR-10 seeds
CIFAR-10	stats-only no-noise	mobile	28.33±1.73	0.1860±0.0208	0.0197	0.0000	41.09	16.30	non-private diagnostic	non-private summary diagnostic; three CIFAR-10 seeds
CIFAR-10	transfer no-noise	mobile	28.20±1.86	0.1800±0.0237	0.0197	0.0199	41.45	16.44	non-private diagnostic	non-private transfer diagnostic; three CIFAR-10 seeds
CIFAR-10	count-only transfer	mobile	28.73±2.45	0.1876±0.0275	0.0002	0.0003	41.44	16.43	81.89	transfer package without prototypes; three CIFAR-10 seeds
CIFAR-10	Stats-only	mobile/no_deficit	28.73±2.45	0.1876±0.0275	0.0197	0.0000	41.09	16.30	191.71	full-summary scheduler without released-count deficit; three CIFAR-10 seeds
CIFAR-10	stats-transfer-M	mobile/no_deficit	28.48±2.64	0.1811±0.0281	0.0197	0.0199	41.45	16.44	191.71	mobile transfer without released-count deficit; three CIFAR-10 seeds
CIFAR-10	stats-transfer-R	random/no_deficit	32.82±3.17	0.2133±0.0305	0.0197	0.0199	79.83	19.28	126.69	random transfer without released-count deficit; three CIFAR-10 seeds

[5] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. ICML*, 2020, pp. 5132–5143.

[6] J. Zhang *et al.*, "FedALA: Adaptive local aggregation for personalized federated learning," in *Proc. AAAI*, 2023, pp. 11237–11244.

[7] K. Pfeiffer, M. Rapp, R. Khalili, and J. Henkel, "Federated learning for computationally constrained heterogeneous devices: A survey," *ACM Comput. Surv.*, vol. 55, no. 14s, pp. 1–27, Jul. 2023.

[8] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surv. Tuts.*, vol. 23, no. 3, pp. 1622–1658, 2021.

[9] H. Kang, S. Cha, J. Shin, J. Lee, and J. Kang, "NeFL: Nested model scaling for federated learning with system heterogeneous clients," *IEEE Trans. Mobile Comput.*, vol. 24, no. 8, pp. 6734–6746, Aug. 2025.

[10] A. Afonin and S. P. Karimireddy, "Towards model agnostic federated learning using knowledge distillation," in *Proc. ICLR*, 2022.

[11] S. Horváth, S. Laskaridis, M. Almeida, I. Leontiadis, S. Venieris, and N. Lane, "FjORD: Fair and accurate federated learning under heterogeneous targets with Ordered Dropout," in *Proc. NeurIPS*, 2021, pp. 12876–12889.

[12] M. Kim, S. Yu, S. Kim, and S.-M. Moon, "DepthFL: Depthwise federated learning for heterogeneous clients," in *Proc. ICLR*, 2023.

[13] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in *Proc. ICLR*, 2021.

[14] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *Proc. NeurIPS*, 2020, pp. 2351–2363.

[15] W. Huang, M. Ye, and B. Du, "Learn from others and be yourself in heterogeneous federated learning," in *Proc. CVPR*, 2022, pp. 10133–10143.

[16] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," arXiv:1910.03581, 2019.

[17] L. Yi, G. Wang, X. Liu, Z. Shi, and H. Yu, "FedGH: Heterogeneous federated learning with generalized global header," arXiv:2303.13137, 2023.

[18] Y. Tan *et al.*, "FedProto: Federated prototype learning across heterogeneous clients," in *Proc. AAAI*, 2022, pp. 8432–8440.

[19] P. P. Liang, T. Liu, L. Ziyin, R. Salakhutdinov, and L.-P. Morency, "Think locally, act globally: Federated learning with local and global representations," arXiv:2001.01523, 2020.

[20] T. Shen *et al.*, "Federated mutual learning," arXiv:2006.16765, 2020.

[21] J. Wang *et al.*, "Towards personalized federated learning via heterogeneous model reassembly," arXiv:2308.08643, 2023.

[22] M. Rasouli, T. Sun, and R. Rajagopal, "FedGAN: Federated generative adversarial networks for distributed data," arXiv:2006.07228, 2020.

[23] Y. Wu, Y. Kang, J. Luo, Y. He, and Q. Yang, "FedCG: Leverage conditional GAN for protecting privacy and maintaining competitive performance in federated learning," arXiv:2111.08211, 2021.

[24] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. ICML*, 2021, pp. 12878–12889.

[25] L. Zhang, L. Shen, L. Ding, D. Tao, and L. Duan, "Fine-tuning global model via data-free knowledge distillation for non-IID federated learning," in *Proc. CVPR*, 2022, pp. 10164–10173.

[26] J. Jang, H. Ha, D. Jung, and S. Yoon, "FedClassAvg: Local representation learning for personalized federated learning on heterogeneous neural networks," in *Proc. ICPP*, 2022, pp. 1–10.

[27] J. Mori, I. Teranishi, and R. Furukawa, "Continual horizontal federated learning for heterogeneous data," in *Proc. IJCNN*, 2022, pp. 1–8.

[28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[29] A. Krizhevsky, V. Nair, and G. Hinton, "Learning multiple layers of features from tiny images," Master's thesis, Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, 2009.

[30] M. Abadi *et al.*, "Deep learning with differential privacy," in *Proc. ACM CCS*, 2016, pp. 308–318.

[31] M. Bun and T. Steinke, "Concentrated differential privacy: Simplifications, extensions, and lower bounds," in *Proc. TCC*, 2016, pp. 635–658.

[32] L. Xie, K. Lin, S. Wang, F. Wang, and J. Zhou, "Differentially private generative adversarial network," arXiv:1802.06739, 2018.

[33] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. ECCV*, 2014, pp. 818–833.

[34] A. Mahendran and A. Vedaldi, "Understanding deep image representations by inverting them," in *Proc. CVPR*, 2015, pp. 5188–5196.

[35] A. Dosovitskiy and T. Brox, "Inverting visual representations with convolutional networks," in *Proc. CVPR*, 2016, pp. 4829–4837.

[36] J. Li *et al.*, "Blockchain assisted decentralized federated learning (BLADE-FL): Performance analysis and resource allocation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 10, pp. 2401–2415, Oct. 2022.